

Portfolio Project

Alexander Ricciardi

Colorado State University Global

ITS410: Database Management

Dr. Murthy Rallapalli

June 8, 2025

Portfolio Project

This documentation is part of the Portfolio Project from ITS410: Database Management at Colorado State University Global. The project consists of two parts, part 1, a lesson learned reflection report, and part 2, a database design and analysis.

Part 1: Lessons Learned and Reflection

The ITS410: Data Management course at Colorado State University Global was a challenging and very satisfying course. I really enjoyed learning about Rational Data Base Management System (RDBMS), more specifically data management using MySQL and MySQL workbench. I learn valuable lessons from the fundamental concepts of data management to more advanced concepts such as tuning and securing relational database using MySQL. This essay provides an overview of what I learn from each module of the ITS410 course and a reflection on how these lessons provided me with skills to further my endeavor .

Lesson Learn

In module 1, "Getting Started with MySQL Workbench," I learn about relational database concepts such as tables, unique keys, and data relationships. I also learn about Structured Query Language (SQL) use for data manipulation and management within RDBSM. Additionally, I spend a significant amount of time learning about and installing MySQL and MySQL Workbench, and then utilizing Workbench to create initial databases and load data using provided SQL scripts.

In module 2, "MySQL Data Types and Retrieve Data," I learned about the different data types use by MySQL to store data. The module also introduced me to the data retrieval statement: the `SELECT` query, and its clauses like `FROM`, `WHERE`, `ORDER BY`, and `LIMIT` used to fetch and organize data within tables. In the next module, module 3, "Retrieve Data from Multiple Tables," I learned concepts for querying and combining data from multiple tables using table and column aliases for improving code readability. As well as the SQL `JOIN` commands and their various types such as `INNER JOIN` for matching rows, `LEFT JOIN` and `RIGHT JOIN` for including non-matching rows from one table, `CROSS JOIN` for Cartesian products, and `SELF JOIN` for relating rows within the same table.

The module 4, "Create Tables, Insert, Update, and Import Data," introduced me to the Data Definition Language (DDL) and Data Manipulation Language (DML) operations which are part of SQL. Operations such as the `CREATE TABLE` statement combined with clauses like `NOT NULL`, `DEFAULT` values, `AUTO_INCREMENT`, and primary keys. The module also introduced me to `INSERT` statement for adding single or multiple rows, to `UPDATE` command for modifying existing records and the `WHERE` clause to specify the conditions that determines which rows should be updated. Following module 4, module 5, "Grouping and Summarizing Data," introduced me to data analysis and summarization. I learned how to use the `GROUP BY` clause in conjunction with aggregate functions like `SUM`, `COUNT`, and `AVG` to generate data summaries. It also explained how to use the `HAVING` clause to filter grouped summaries. The module also introduced me to methods for combining distinct query results vertically using the `UNION` and `INTERSECT` operators, as well as the concept of subqueries to performed complex data retrieval.

In module 6, "Stored Procedures and Functions," I learned how to create and use stored procedures and user-defined functions, which allow the implementation of reusable SQL code directly within the database. I learned about their syntax used to create, call, alter, and drop these stored procedures. Additionally, I learned about their benefits such as improving application security, modularity, reduced

network traffic, reusability, and functionality for executing complex operations. The next module, module 7, “Database Tuning and Security,” introduced me to database tuning, backup, and recovery. The module explained the importance of database indexes, particularly when using MySQL's B-tree indexing structure for searching and fetching data. It also demonstrated how to use statements such as `EXPLAIN`, and considering object characteristics like table size to analysis queries for optimization purposes. Additionally, it described the best practices for securing a database.

The final module, module 8, “Backup and Recovery,” introduced me to database backup and recovery best practices. It described how these practices are crucial for preventing catastrophic data losses. I learned how to perform logical database backups by using the `mysqldump` program utility, which creates SQL script files that can be used to recreate the database schemas and data. I also learned about physical backups, which are direct copy of the database files and directories. Finally, I learned about database replication, which is a live slave copies of a master database that can live in one or more slave servers.

Reflection

I enjoyed learning about data management in MySQL; the course provided me with a robust foundation for data management and data system analysis. Additionally, what I learned through this course give me the skills needed to choose data types for a specific data application, to create and modify databases, how to modify and manage those databases, how to administered data management system-based MySQL, how to keep those systems secure, how to protect them from catastrophic data losses, and how to maintain and optimize these systems. All these skills that I acquired throughout this course are crucial for effectively managing and administrate data systems, which is an essential component of information technology and businesses functionality. These acquired skills provide me with a practical toolkit that is essential for my future endeavors as a computer science student and as a computer science professional. They will allow me to confidently design, implement, and maintain RDBMS, optimize my queries, implement data security measures and maintain data integrity within those systems.

Conclusion

The ITS410: Data Management course at Colorado State University Global provided me with fundamentals knowledge and understanding of data management, more specifically Rational Data Base Management System (RDBMS). In this course I learned how to use MySQL and MySQL Workbench to create, modify, manage, and administer complex rational databases. This provided me with the knowledge, critical thinking skills, and the technical skills that I need as a computer science student and professional to tackle real-world database challenges and contribute to the development of database projects. Furthermore, I enjoy the course and believe that the skills and knowledge that I acquired will be immensely beneficial in my future academic and professional endeavors in computer science.

Part 2: Queries

The Assignment Direction:

Using the My Guitar Shop database you installed in Module 1, develop the following queries.

SUBMIT A SCREENSHOT OF EACH STEP.

1. Write a SELECT statement that returns these column names and data from the Products table:

product_name	The product_name column
list_price	The list_price column
discount_percent	The discount_percent column
discount_amount	A column that's calculated from the previous two columns
discount_price	A column that's calculated from the previous three columns

 - Round the discount_amount and discount_price columns to two decimal places.
 - Sort the result set by the discount_price column in descending sequence.
 - Use the LIMIT clause so the result set contains only the first five rows.
 - Submit a screenshot.

2. Write a SELECT statement that returns these column names and data from the Order_Items table:

item_id	The item_id column
item_price	The item_price column
discount_amount	The discount_amount column
quantity	The quantity column
price_total	A column that's calculated by multiplying the item price by the quantity
discount_total	A column that's calculated by multiplying the discount amount by the quantity
item_total	A column that's calculated by subtracting the discount amount from the item price and then multiplying by the quantity

 - Only return rows where the item_total is greater than 500.
 - Sort the result set by the item_total column in descending sequence.
 - Submit a screenshot.

3. Write a SELECT statement that returns the product_name and list_price columns from the Products table.
 - Return one row for each product that has the same list price as another product.
 - Hint: Use a self-join to check that the product_id columns aren't equal but the list_price columns are equal.
 - Sort the result set by the product_name column. Submit a screenshot.

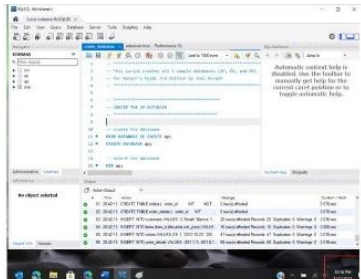
4. Write a SELECT statement that returns these two columns:

category_name	The category_name column from the Categories table
product_id	The product_id column from the Products table

 - Return one row for each category that has never been used.
 - Hint: Use an outer join and only return rows where the product_id column contains a null value.
 - Submit a screenshot.

5. Write an INSERT statement that adds this row to the Customers table:
 - email_address: rick@raven.com
 - password: (empty string)
 - first_name: Rick
 - last_name: Raven
 - Use a column list for this statement.
 - Submit a screenshot.
6. Write a SELECT statement that answers this question: Which customers have ordered more than one product?
 - Return these columns:
 - The email_address column from the Customers table
 - The count of distinct products from the customer's orders
 - Sort the result set in ascending sequence by the email_address column.
 - Submit a screenshot.
7. Write a SELECT statement that answers this question: What is the total quantity purchased for each product within each category?
 - Return these columns
 - The category_name column from the category table
 - The product_name column from the products table
 - The total quantity purchased for each product with orders in the Order_Items table
 - Use the WITH ROLLUP operator to include rows that give a summary for each category name as well as a row that gives the grand total.
 - Use the IF and GROUPING functions to replace null values in the category_name and product_name columns with literal values if they're for summary rows.
 - Submit a screenshot.
8. Write and execute a script that creates a user with a username using your firstname initial and lastname and password of your choosing. This user should be able to connect to MySQL from any computer.
 - This user should have SELECT, INSERT, UPDATE, and DELETE privileges for the Customers, Addresses, Orders, and Order_Items tables of the My Guitar Shop database.
 - However, this user should only have SELECT privileges for the Products and Categories tables.
 - Also, this user should not have the right to grant privileges to other users.
 - Check the privileges for the user by using the SHOW GRANTS statement.
 - Submit a screenshot.

All the screenshots should show current date. Example of screenshot.



Submit your labeled results screenshots in a Word file.

Step 1: Write a SELECT statement that returns these column names and data from the Products table.

Figure 1

Assignment Steps 1

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```

1  SELECT
2      product_name,
3      list_price,
4      discount_percent,
5      ROUND((list_price * discount_percent / 100), 2) AS discount_amount,
6      ROUND((list_price - (list_price * discount_percent / 100)), 2) AS discount_price
7  FROM
8      products
9  ORDER BY
10     discount_price DESC
11  LIMIT 5;
12

```

The Results window displays the following data:

product_name	list_price	discount_percent	discount_amount	discount_price
Gibson SG	2517.00	52.00	1308.84	1208.16
Gibson Les Paul	1199.00	30.00	359.70	839.30
Yamaha DGX 640 88-Key Digital Piano	799.99	0.00	0.00	799.99
Tama 5-Piece Drum Set with Cymbals	799.99	15.00	120.00	679.99
Fender Precision	799.99	30.00	240.00	559.99

The Output window shows the following message:

```

#    Time    Action
1  09:26:40  SELECT product_name, list_price, discount_percent, ROU... 5 row(s) returned

```

Note: The figure illustrates the MySQL Workbench result after performing steps 1.

Please see the next page.

Step 2: Write a SELECT statement that returns these column names and data from the Order_Items.

Figure 2
Assignment Steps 2

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```

1  SELECT
2      item_id,
3      item_price,
4      discount_amount,
5      quantity,
6      (item_price * quantity) AS price_total,
7      (discount_amount * quantity) AS discount_total,
8      ((item_price - discount_amount) * quantity) AS item_total
9  FROM
10     order_items
11  WHERE
12     ((item_price - discount_amount) * quantity) > 500
13  ORDER BY
14     item_total DESC;
15

```

The Results window displays the following data:

	item_id	item_price	discount_amount	quantity	price_total	discount_total	item_total
5	1199.00	359.70	2	2398.00	719.40	1678.60	
3	2517.00	1308.84	1	2517.00	1308.84	1208.16	
1	1199.00	359.70	1	1199.00	359.70	839.30	
11	799.99	120.00	1	799.99	120.00	679.99	
9	799.99	240.00	1	799.99	240.00	559.99	

The Output window shows the following message:

```

#    Time    Action
1  14:17:19  SELECT item_id, item_price, discount_amount, quantity, ... 5 row(s) returned

```

The duration of the query execution is 0.000 sec / 0.000 sec.

Note: The figure illustrates the MySQL Workbench result after performing steps 2.

Please see the next page.

Step 3: Write a SELECT statement that returns the product_name and list_price columns from the Products table.

Figure 3
Assignment Step 3

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```

1 SELECT DISTINCT
2     p1.product_name,
3     p1.list_price
4 FROM
5     products p1 -- products table version 1
6 -- self-join
7 INNER JOIN
8     -- p2 is products table version 2
9     products p2 ON p1.list_price = p2.list_price AND p1.product_id <> p2.product_id
10 ORDER BY
11     p1.product_name;
12

```

The Results window displays the following data:

product_name	list_price
Fender Precision	799.99
Tama 5-Piece Drum Set with Cymbals	799.99
Yamaha DGX 640 88-Key Digital Piano	799.99

The Output window shows the execution details:

#	Time	Action	Message	Duration / Fetch
1	14:42:41	SELECT DISTINCT p1.product_name, p1.list_price FROM ...	3 row(s) returned	0.000 sec / 0.000 sec

Note: The figure illustrates the MySQL Workbench result after performing steps 3.

Please see the next page.

Step 4: Write a SELECT statement that returns these two columns : category_name and product_id.

Figure 4

Assignment Step 4

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```

1  SELECT
2      c.category_name,
3      p.product_id
4  FROM
5      categories c
6  LEFT JOIN
7      products p ON c.category_id = p.category_id
8  WHERE
9      p.product_id IS NULL;

```

The Results window displays the following data:

category_name	product_id
Saxophone	NULL
Trumpet	NULL

The Output window shows the execution details:

#	Time	Action	Message	Duration / Fetch
1	14:51:47	SELECT	c.category_name, p.product_id FROM ... 2 row(s) returned	0.000 sec / 0.000 sec

Note: The figure illustrates the MySQL Workbench result after performing steps 4.

Please see the next page.

Step 5: Write an INSERT statement that adds this row to the Customers table.

Figure 5
Assignment Step 5

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```

1  INSERT INTO customers (
2      email_address,
3      password,
4      first_name,
5      last_name
6  )
7  VALUES (
8      'rick@raven.com',
9      '',
10     'Rick',
11     'Raven'
12 );
13
14 SELECT * FROM customers;
15

```

The Result Grid shows the output of the query. The table has 5 columns: customer_id, email_address, password, first_name, and last_name. The data is as follows:

customer_id	email_address	password	first_name	last_name
1	allan.sherwood@yahoo.com	650215acac746f0e32bdf387439eefc1358737	Allan	Sherwood
2	barryz@gmail.com	3f563468d42a448cb1e56924529f6e7bbe529cc7	Barry	Zimmer
3	christineb@solarone.com	ed19f5c0833094026a2f1e9e6f08a35d26037066	Christine	Brown
4	david.goldstein@hotmail.com	b444ac06613fc8d63795be9ad0beaf55011936ac	David	Goldstein
5	erinv@gmail.com	109f4b3c50d7b0df729d299bc6f8e9ef9066971f	Erin	Valentino
6	frankwilson@sbcglobal.net	3ebfa301dc59196f18593c45e519287a23297589	Frank Lee	Wilson
7	gary_hernandez@yahoo.com	1ff2b3704aede04eeeb51e50ca698efd50a1379b	Gary	Hernandez
8	heatheresway@mac.com	911ddc3b8f9a13b5499b6bc4638a2b4f3f68bf23	Heather	Esway
9	rick@raven.com		Rick	Raven
*	NULL	NULL	NULL	NULL

The Output panel shows the execution log:

#	Time	Action	Message	Duration / Fetch
5	15:16:00	SELECT * FROM customers LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
6	15:16:19	INSERT INTO customers (email_address, password, fir...	1 row(s) affected	0.000 sec
7	15:16:19	SELECT * FROM customers LIMIT 0, 1000	9 row(s) returned	0.000 sec / 0.000 sec

Note: The figure illustrates the MySQL Workbench result after performing steps 5.

Please see the next page.

Step 6: Write a SELECT statement that answers this question: Which customers have ordered more than one product?

Figure 6
Assignment Step 6

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```

1  SELECT
2      c.email_address,
3      COUNT(oi.product_id) AS products_count
4  FROM
5      customers c
6  INNER JOIN
7      orders o ON c.customer_id = o.customer_id
8  INNER JOIN
9      order_items oi ON o.order_id = oi.order_id
10 GROUP BY
11     c.customer_id, c.email_address
12 HAVING
13     COUNT(oi.product_id) > 1
14 ORDER BY
15     c.email_address ASC;
16

```

The Result Grid shows the following data:

email_address	products_count
allan.sherwood@yahoo.com	3
david.goldstein@hotmail.com	2
frankwilson@sbcglobal.net	3

The Output panel shows the following message:

```

1 15:36:53 SELECT c.email_address, COUNT(oi.product_id) AS product... 3 row(s) returned
0.000 sec / 0.000 sec

```

Note: The figure illustrates the MySQL Workbench result after performing steps 6. It shows the emails of customers that ordered more than one product and the number of products they ordered.

Please see the next page.

Step 7: Write a SELECT statement that answers this question: What is the total quantity purchased for each product within each category?

Figure 7
Assignment Step 7

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```

1 SELECT
2     IF(GROUPING(c.category_name) = 1,
3         'All Categories - Grand Total',
4         c.category_name) AS category_name,
5     IF(GROUPING(p.product_name) = 1,
6         IF(GROUPING(c.category_name) = 1,
7             '', -- Product name, blank for the grand total row
8             '----- Category Total -----'), -- category subtotal
9         p.product_name) AS product_name,
10    SUM(oi.quantity) AS total_quantity_purchased
11 FROM
12     categories c
13 INNER JOIN
14     products p ON c.category_id = p.category_id
15 INNER JOIN
16     order_items oi ON p.product_id = oi.product_id
17 GROUP BY
18     c.category_name, p.product_name WITH ROLLUP;

```

The Results window displays the following data:

category_name	product_name	total_quantity_purchased
Basses	Fender Precision	1
Basses	----- Category Total -----	1
Drums	Ludwig 5-piece Drum Set with Cymbals	1
Drums	Tama 5-Piece Drum Set with Cymbals	1
Drums	----- Category Total -----	2
Guitars	Fender Stratocaster	2
Guitars	Gibson Les Paul	3
Guitars	Gibson SG	1
Guitars	Rodriguez Caballero 11	1
Guitars	Washburn D10S	2
Guitars	Yamaha FG700S	1
Guitars	----- Category Total -----	10
All Categories - Grand Total		13

The Output window shows the execution message: "SELECT IF(GROUPING(c.category_name) = 1, 'All Categories - Gr... 13 row(s) returned".

Note: The figure illustrates the MySQL Workbench result after performing steps 7.

Please see the next page.

Step 8: Write and execute a script that creates a user with a username using your firstname initial and lastname and password of your choosing.

Figure 8
Assignment Step 8

The screenshot shows the MySQL Workbench interface. The SQL Editor contains a script to create a user 'ar' and grant various privileges. The Results window shows the output of the script, and the Output window shows the execution details.

SQL Script:

```

1 CREATE USER IF NOT EXISTS 'ar'@'%' IDENTIFIED BY 'password123';
2
3 -- Grant SELECT, INSERT, UPDATE, DELETE privileges for tables (listed below) in the My Guitar Shop database
4 GRANT SELECT, INSERT, UPDATE, DELETE ON my_guitar_shop.Customers TO 'ar'@'%';
5 GRANT SELECT, INSERT, UPDATE, DELETE ON my_guitar_shop.Addresses TO 'ar'@'%';
6 GRANT SELECT, INSERT, UPDATE, DELETE ON my_guitar_shop.Orders TO 'ar'@'%';
7 GRANT SELECT, INSERT, UPDATE, DELETE ON my_guitar_shop.Order_Items TO 'ar'@'%';
8
9 -- Grant SELECT only privileges for Products and Categories tables
10 GRANT SELECT ON my_guitar_shop.Products TO 'ar'@'%';
11 GRANT SELECT ON my_guitar_shop.Categories TO 'ar'@'%';
12
13 -- Note that by default, a user cannot grant privileges to others
14 -- The GRANT OPTION privilege can be explicitly granted to users using the WITH GRANT OPTION
15 -- However, the user 'ar' does not need/have the right to grant privileges
16 -- The REVOKE statement below is an extra explicit safeguard (optional)
17 REVOKE GRANT OPTION ON *.* FROM 'ar'@'%';
18
19 SHOW GRANTS FOR 'ar'@'%';
20

```

Result Grid:

Grants for ar@%
GRANT USAGE ON *.* TO 'ar'@'%'
GRANT SELECT, INSERT, UPDATE, DELETE ON 'my_guitar_shop'. 'addresses' TO 'ar'@'%'
GRANT SELECT ON 'my_guitar_shop'. 'categories' TO 'ar'@'%'
GRANT SELECT, INSERT, UPDATE, DELETE ON 'my_guitar_shop'. 'customers' TO 'ar'@'%'
GRANT SELECT, INSERT, UPDATE, DELETE ON 'my_guitar_shop'. 'order_items' TO 'ar'@'%'
GRANT SELECT, INSERT, UPDATE, DELETE ON 'my_guitar_shop'. 'orders' TO 'ar'@'%'
GRANT SELECT ON 'my_guitar_shop'. 'products' TO 'ar'@'%'

Output:

#	Time	Action	Message	Duration / Fetch
1	17:20:15	CREATE USER IF NOT EXISTS 'ar'@'%' IDENTIFIED BY 'password123';	0 row(s) affected	0.000 sec
2	17:20:15	GRANT SELECT, INSERT, UPDATE, DELETE ON my_guitar_shop...	0 row(s) affected	0.000 sec
3	17:20:15	GRANT SELECT, INSERT, UPDATE, DELETE ON my_guitar_shop...	0 row(s) affected	0.000 sec
4	17:20:15	GRANT SELECT, INSERT, UPDATE, DELETE ON my_guitar_shop...	0 row(s) affected	0.000 sec
5	17:20:15	GRANT SELECT, INSERT, UPDATE, DELETE ON my_guitar_shop...	0 row(s) affected	0.016 sec
6	17:20:15	GRANT SELECT ON my_guitar_shop.Products TO 'ar'@'%'	0 row(s) affected	0.000 sec
7	17:20:15	GRANT SELECT ON my_guitar_shop.Categories TO 'ar'@'%'	0 row(s) affected	0.000 sec
8	17:20:15	REVOKE GRANT OPTION ON *.* FROM 'ar'@'%'	0 row(s) affected	0.000 sec
9	17:20:15	SHOW GRANTS FOR 'ar'@'%'	7 row(s) returned	0.000 sec / 0.000 sec

Note: The figure illustrates the MySQL Workbench result after performing steps 8.